chapter 05: Performance considerations

This chapter explored **GPU off-chip memory (DRAM) architecture** and key performance aspects like **global memory access coalescing** and **memory latency hiding through parallelism**. We introduced **thread granularity coarsening** as a crucial optimization technique. These insights, combined with previous chapters, enable understanding of kernel code performance. The chapter concluded with a **checklist of common performance optimizations**, setting the foundation for upcoming parallel computation patterns and application case studies.



Terminology	Explanation
Memory Coalescing	A technique for combining multiple memory accesses into a single transaction, thereby reducing global memory traffic. It is achieved when threads in a warp access adjacent memory locations. This is important

Terminology	Explanation
	because global memory accesses are typically much slower than other operations.
DRAM	Dynamic Random-Access Memory, the off-chip memory used in GPUs for storing large amounts of data. It is slower to access than on-chip memories like shared memory and registers. Chapter 6 briefly discusses DRAM architecture and performance considerations.
Memory Latency	The time it takes to access data from memory. It can be a significant bottleneck in program performance, especially when accessing DRAM. GPUs use various techniques to hide memory latency, such as executing multiple warps concurrently.
Memory Parallelism	The ability to perform multiple memory accesses in parallel. This can help to hide memory latency. GPUs achieve high memory parallelism by having multiple memory controllers and using techniques like memory coalescing.
Thread Granularity Coarsening	A technique for reducing the overhead of parallelism by assigning multiple units of work to each thread. This can be beneficial when the overhead of parallelism (e.g., thread creation, synchronization) is greater than the performance benefit of parallelism. Coarsening reduces the number of threads, which can also help to reduce contention for shared resources like shared memory.
Occupancy	A measure of how well the computational resources of an SM are being utilized. It is the ratio of the number of active warps on an SM to the maximum number of warps that can be active. Maximizing occupancy is crucial for hiding latency and achieving high performance.

5.1 Memory coalescing

- **Global Memory Performance in CUDA Kernels**
  - Global memory bandwidth can become a **performance bottleneck**
  - CUDA applications process **massive amounts of data** in parallel
  - Need efficient techniques to optimize memory access
- **Memory Optimization Techniques**
  - **Tiling Techniques**
    - Uses shared memory
    - Reduces total global memory access needed
  - **Memory Coalescing**
    - definition:
      - Memory coalescing in CUDA programming refers to a set of techniques and strategies that optimize global memory access patterns by **ensuring that memory accesses from multiple threads in a warp (32 threads) are grouped together into as few memory transactions as possible**. This optimization minimizes memory latency and maximizes memory bandwidth utilization, which is crucial for achieving high performance on GPUs.
    - example:

Coalesced Access:

cuda 复制代码

```
__global__ void coalescedAccess(float *data) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;  // Contiguous indices
    data[idx] = data[idx] * 2.0f;
}
```

- Each thread accesses a contiguous memory location ($data[idx]$).

Non-Coalesced Access:

cuda 复制代码

```
__global__ void nonCoalescedAccess(float *data, int stride) {
    int idx = threadIdx.x * stride + blockIdx.x * blockDim.x;
    data[idx] = data[idx] * 2.0f;
}
```

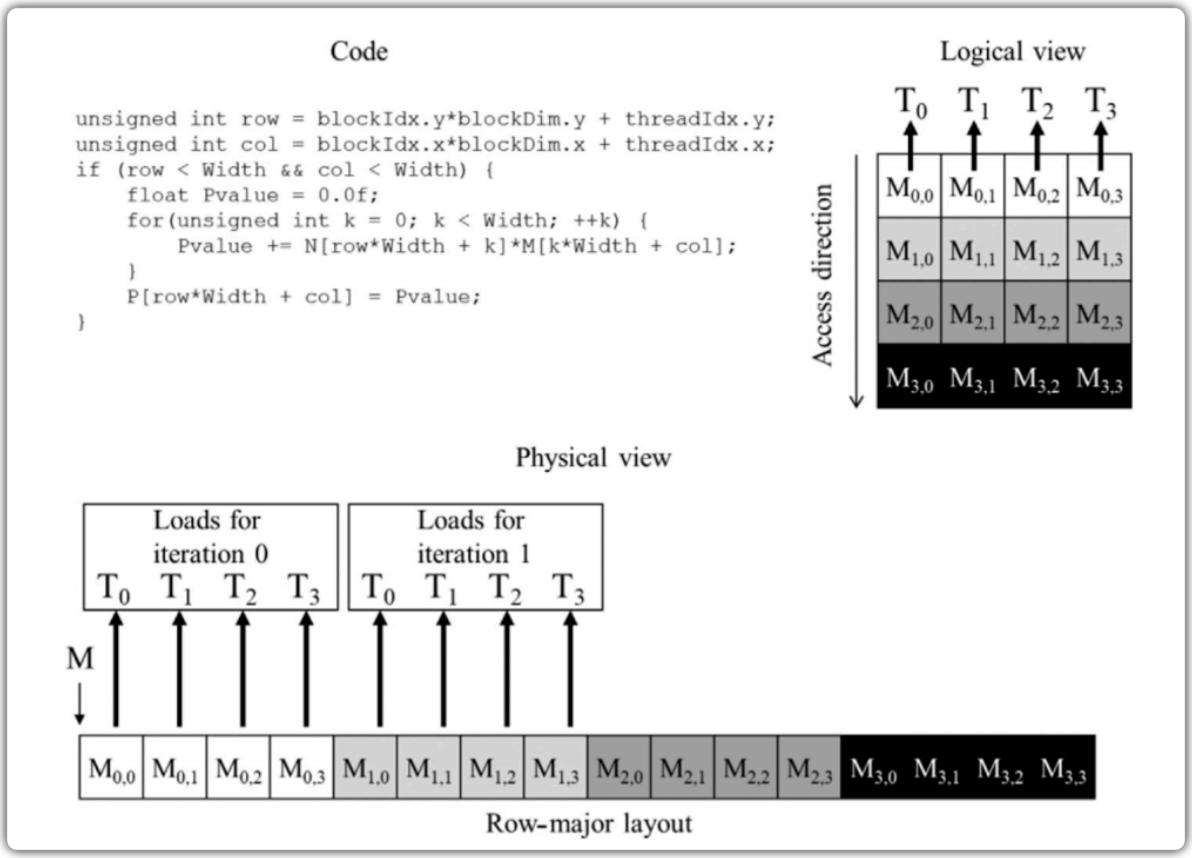
- Threads access memory with a **stride**, leading to scattered memory accesses and multiple transactions.

    - Efficiently moves data between global and shared memory/registers
    - Often used together with tiling
    - Critical for reaching peak performance

- **DRAM Architecture Details**
    - Global memory uses **DRAM cells** (small capacitors)
        - Stores bits as electrical charge
        - Reading process is **relatively slow** (tens of nanoseconds)
        - Much slower than modern computing cycles (subnanosecond)
- **DRAM Performance Enhancement**
    - Uses **parallelism** to increase memory access throughput
    - **DRAM Bursts**
        - Accesses range of consecutive locations at once
        - Multiple sensors work in parallel
        - Data transferred at high speed
        - Applications perform better when using focused data from these bursts
- **Memory Access Efficiency in CUDA**
    - Modern DRAMs use **burst organization**
    - CUDA devices employ techniques for **high global memory access efficiency**
    - Key is organizing memory accesses into **favorable patterns**
- **Warp Execution and Memory Access**

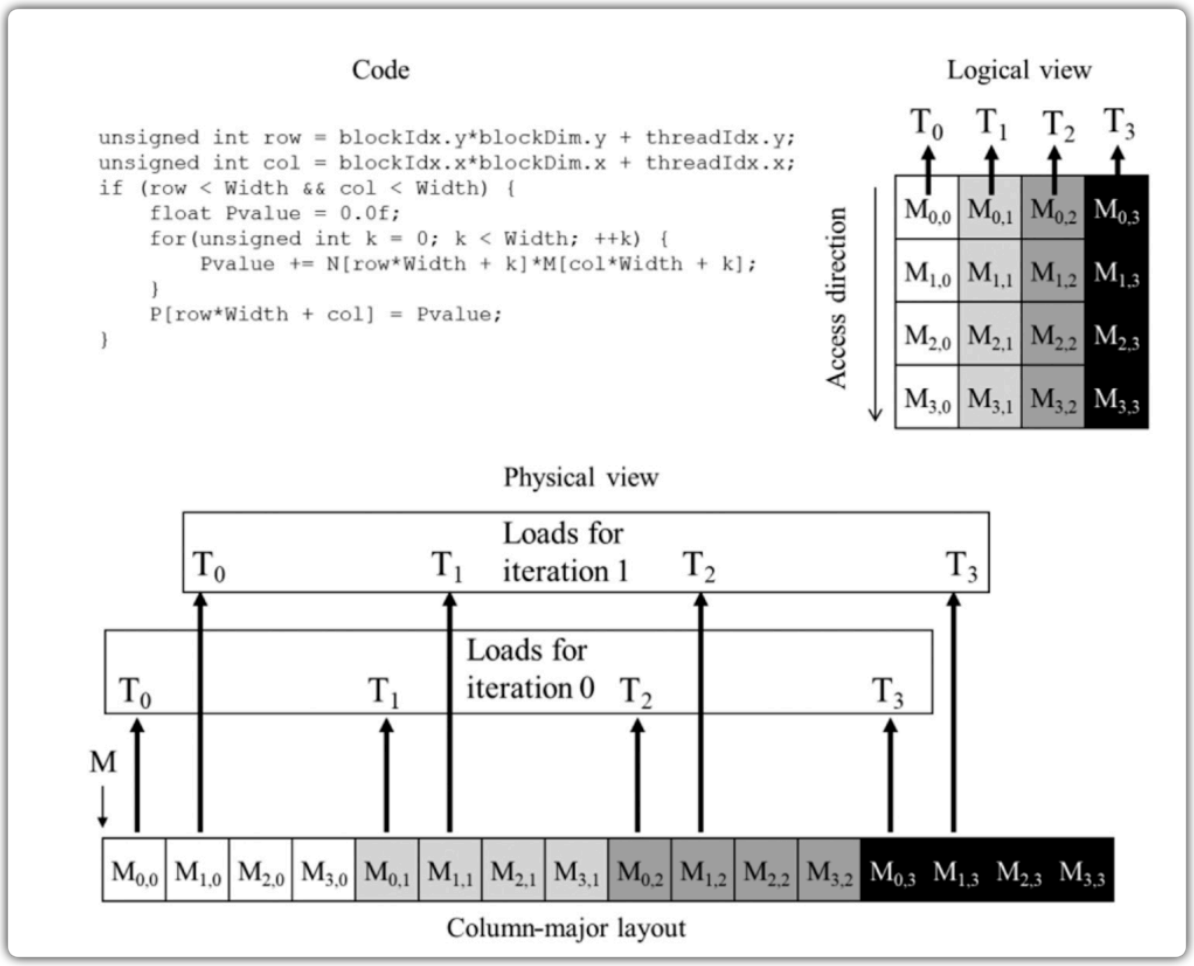
- Threads in a warp execute **same instruction** at any time
- Hardware detects if threads access **consecutive memory locations**
- Most favorable pattern: threads access **consecutive global memory locations**
- Hardware **coalesces** multiple accesses into consolidated DRAM access

- **Example of Coalesced Access**
  - Thread 0 accesses location X
  - Thread 1 accesses location X+1
  - Thread 2 accesses location X+2
  - Result: Combined into **single request** for consecutive locations
  - Enables DRAM to deliver data as a **burst**
- **Memory Address Formation in C Arrays**
  - Uses **row-major convention**
  - Elements placed in **linear address space**
  - Adjacent row elements in **consecutive memory locations**
  - Example: In a matrix
    - Row 0 elements placed first
    - Followed by Row 1 elements
    - Then Row 2, Row 3, etc.
  - Note: M0,0 and M1,0 are **four locations apart** in memory
- **Matrix Multiplication Implementation**
  - Consecutive threads assigned to **consecutive output elements**



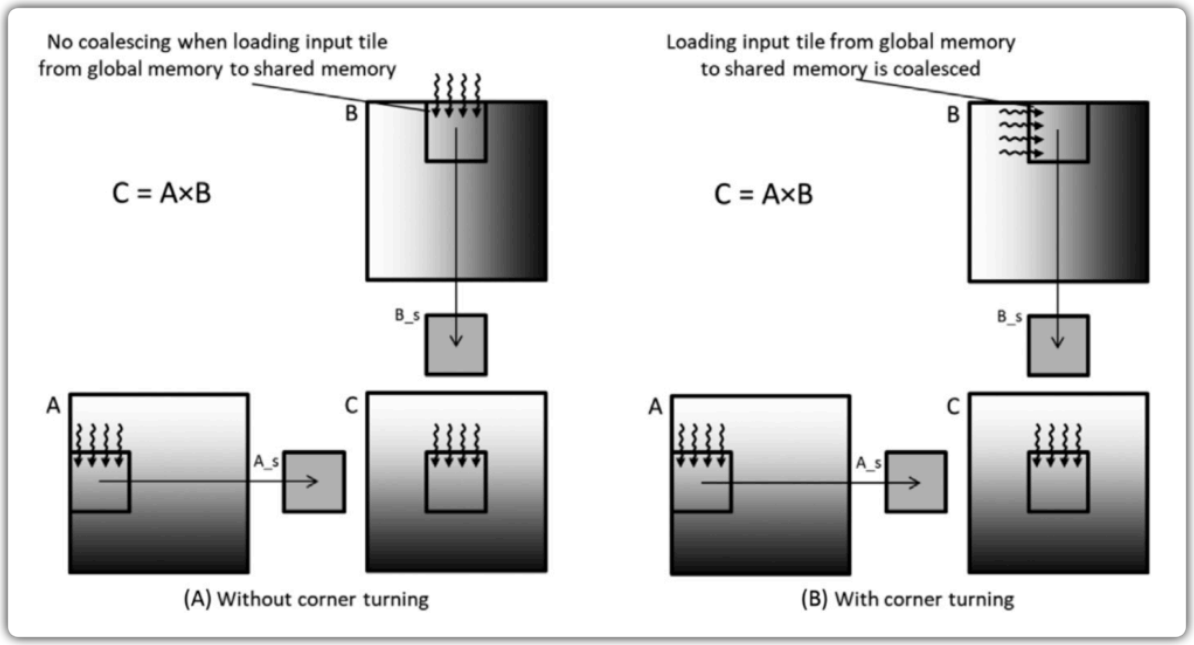
  - Threads iterate through **consecutive columns**

- Access pattern enables **coalescing**
- Array index calculation: k*Width+col
  - k and Width: **same value** across threads
  - col: varies based on **threadIdx.x**
  - Results in **consecutive memory access** pattern
- Physical View of Access Pattern in Row-Major Order
  - In iteration 0:
    - Consecutive threads access **consecutive elements** in row 0
    - Elements are **adjacent in memory**
  - In iteration 1:
    - Similar pattern continues with row 1 elements
  - Result: **Favorable memory access pattern** that can be coalesced
- Column-Major Order Storage Scenario
  - **Reasons for column-major storage**:
    - Matrix multiplication with transposed matrices
    - Avoiding duplicate storage of original and transposed forms
    - Accessing transposed form by switching row/column indices
- Memory Access Pattern in Column-Major Order



  - **Key Differences**:
    - Index calculation: col*Width + k

        - Consecutive threads access elements that are **Width apart**
        - **Not favorable** for coalescing
- **Physical Memory Access Challenges**
    - In iteration 0:
        - Consecutive threads access row 0 elements
        - Elements are **not adjacent** in memory
    - Practical Impact:
        - For large matrices (hundreds/thousands of elements)
        - Elements can be **very far apart**
        - Hardware **cannot coalesce** these accesses
- **Corner Turning Application Example**
    - Input matrix A: stored in **row-major layout** in global memory
    - Input matrix B: stored in **column-major layout** in global memory
    - Output matrix C: stored in **row-major layout** in global memory
    - Shows how **four threads** handle consecutive elements at output tile's top edge

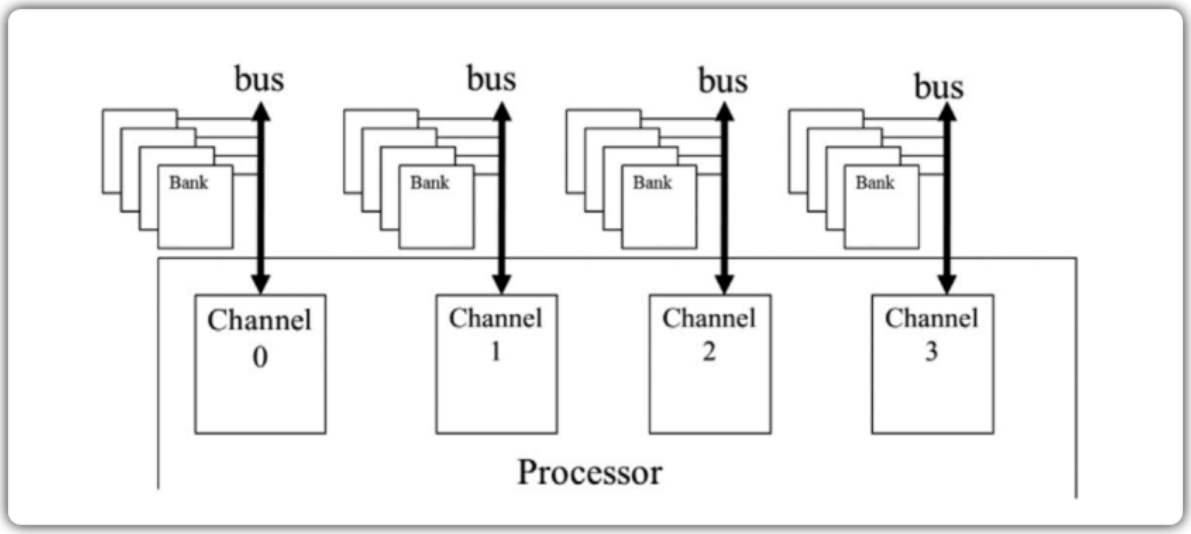


- **Access Pattern for Matrix A**
    - Four threads load top edge elements of input tile
    - **Coalesced access** achieved because:
        - Consecutive threads access consecutive memory elements
        - Elements are adjacent in memory (row-major layout)
- **Access Pattern for Matrix B - Problem**
    - Using Chapter 5 approach leads to issues:
        - Elements loaded by consecutive threads are **far apart** in memory

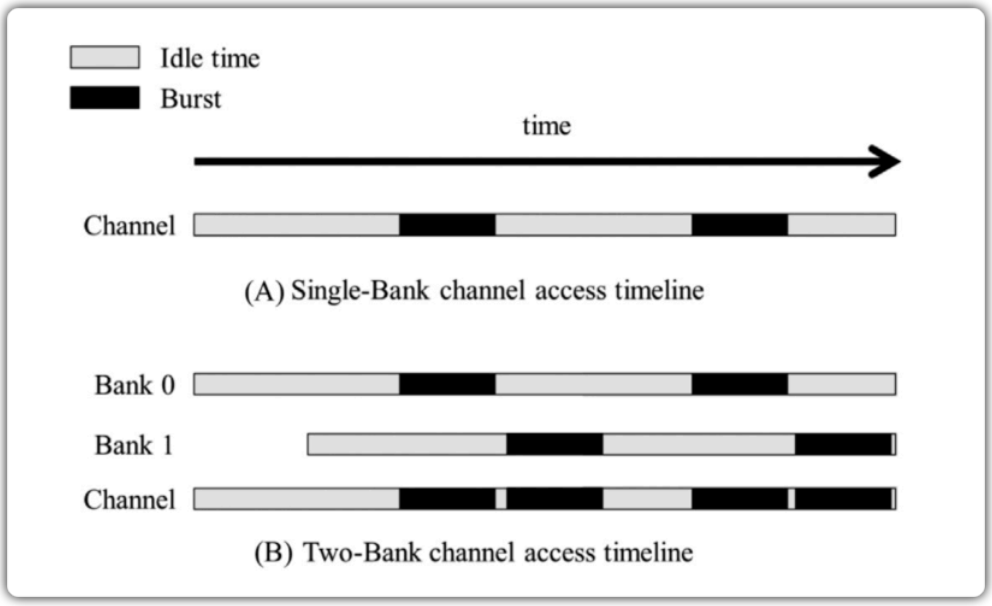
        - Results in **uncoalesced memory access**
        - Inefficient due to column-major layout
- **Solution for Matrix B Access**
    - **Modified approach**: Four consecutive threads load consecutive elements at left edge
    - **Implementation details**:
        - Exchanges roles of threadIdx.x and threadIdx.y
        - Achieves **coalesced memory access**
        - Tile placement in shared memory can be either column-major or row-major
        - Shared memory (SRAM) doesn't require coalescing
- **Benefits of Memory Coalescing**
    - **Reduces global memory traffic**
    - Combines multiple memory accesses into single access
    - Requirements:
        - Simultaneous access timing
        - Adjacent memory locations
- **Traffic Congestion Analogy**
    - Similar to highway traffic congestion
    - **Root cause**: Too many vehicles for road capacity
    - **Effects**:
        - Increased travel time
        - Commute times can double or triple

5.2 Hiding memory latency

- **DRAM Parallel Organization** consists of multiple forms:
    - **Bursting**: Multiple locations accessed in parallel in DRAM core array
    - **Banks and Channels**: Additional parallel organization forms
- **Channel Structure**:
    - Processor contains **one or more channels**



    - Each channel has:
        - Memory controller
        - Bus connecting DRAM banks to processor
    - Typical systems have:
        - **1-8 channels**
        - Multiple banks per channel
- **Bus Data Transfer Bandwidth**:
    - Determined by:
        - Width
        - Clock frequency
    - **DDR (Double Data Rate)**:
        - Two transfers per clock cycle
        - Example: 64-bit DDR bus at 1 GHz = **16GB/s bandwidth**
- **Bandwidth Requirements**:
    - Modern CPU: **~32 GB/s** (needs 2 channels)
    - Modern GPU: **~256 GB/s** (needs 16 channels)
- **Bank Organization**:
    - Number of banks determined by bus bandwidth utilization
    - Each bank contains:
        - DRAM cell array
        - Sensing amplifiers
        - Interface for burst data delivery
- **Single Bank Timing Issues**:



- Long latency for:
  - Decoder cell enablement
  - Cell charge sharing with sensing amplifier
- **Efficiency Problem**:
  - Access latency much longer than data transfer time
  - Example: 20:1 ratio results in only **4.8% utilization**
  - 16 GB/s channel would only deliver **0.76 GB/s**
- **Solution**: Multiple banks per channel
- **Bank Connection and Channel Bus Access**
  - When two banks connect to a channel bus:
    - Second bank can initiate access while first bank serves another
    - Allows overlap of DRAM cell array access latency
- **Two-Bank Organization Timing**
  - Bank 0 starts cell array access first
  - Bank 1 begins shortly after
  - Bank 0 transfers burst data when complete
  - Bank 1 transfers burst data next
  - Pattern repeats for subsequent accesses

- **Channel Bus Bandwidth Utilization**
  - Two banks can potentially **double data transfer bandwidth**

Key Concepts

1. **Cell Array Access Latency (L):** The time required to fetch data from a memory bank.
2. **Data Transfer Time (T):** The time required to transfer the fetched data across the channel.
3. **Ratio (R):** $R = \frac{L}{T}$, indicating how many transfer times fit within the access latency.

For maximum bandwidth utilization, a new bank should be ready to send data as soon as the current bank finishes transferring. This requires overlapping data fetches from multiple banks.

Example

Assume:

- **Cell array access latency (L) = 10 ns**
- **Data transfer time (T) = 2 ns**
- **Ratio** $R = L/T = 10/2 = 5$

According to the rule, we need at least $R + 1 = 5 + 1 = 6$ banks to fully utilize the channel bandwidth.

Process with 6 Banks

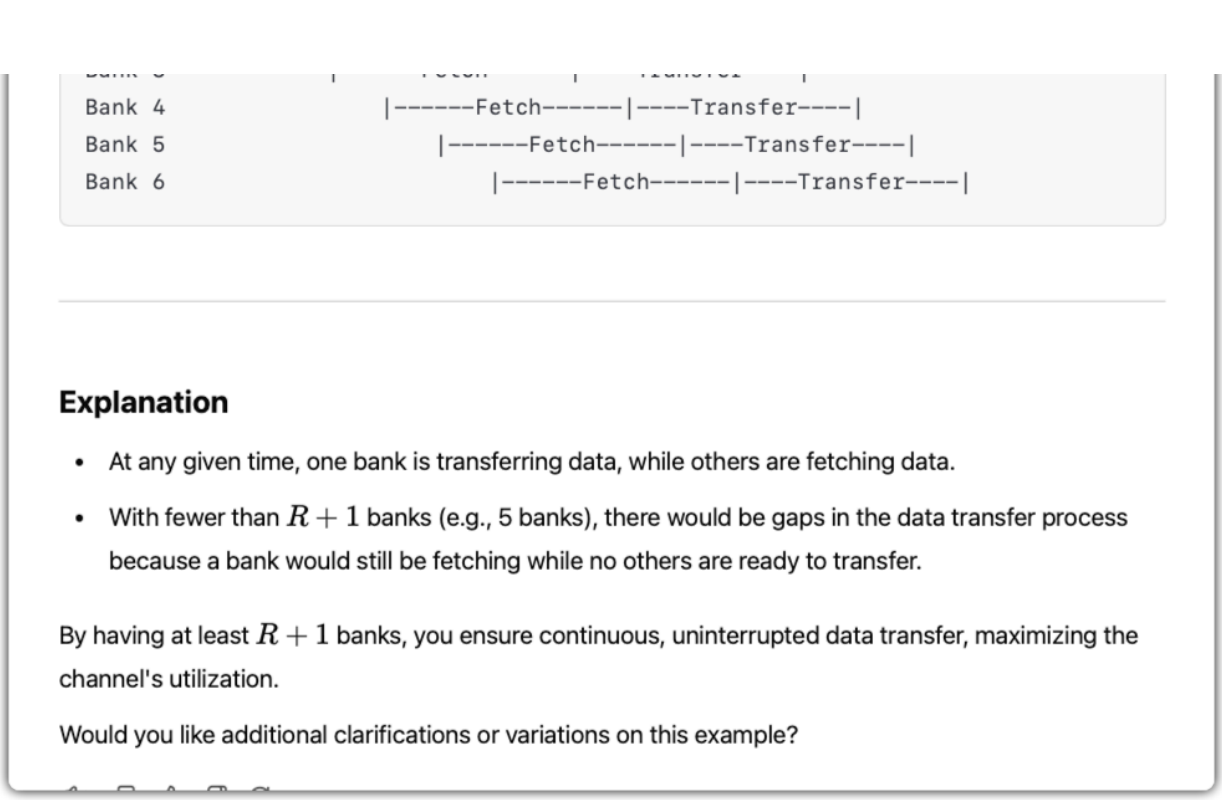
1. **Cycle 1:** Bank 1 starts fetching data (takes 10 ns).
2. **Cycle 2:** Bank 2 starts fetching data.
3. **Cycle 3:** Bank 3 starts fetching data.
4. **Cycle 4:** Bank 4 starts fetching data.
5. **Cycle 5:** Bank 5 starts fetching data.
6. **Cycle 6:** Bank 6 starts fetching data.
7. **Cycle 7:** Bank 1 finishes fetching data and starts transferring.
8. **Cycle 8:** Bank 2 finishes fetching and starts transferring.

Each bank's access latency overlaps with data transfers, ensuring continuous data flow.

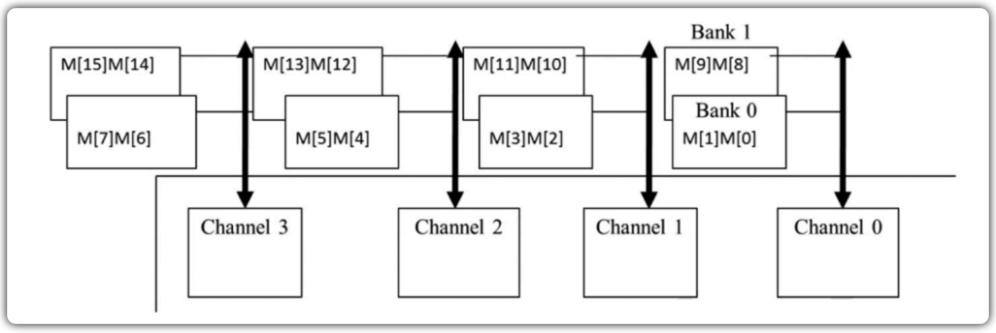
Diagram

plaintext 复制代码

```
Time (ns) ->  0   2   4   6   8  10  12  14  16  18  20
--------------------------------------------------------
Bank 1    |------Fetch------|----Transfer----|
Bank 2        |------Fetch------|----Transfer----|
Bank 3            |------Fetch------|----Transfer----|
```



    - For ratio R (cell array access latency / data transfer time):
        - Need minimum of R+1 banks for full bandwidth utilization
        - Example: If R=20, need at least 21 banks per channel bus
- **Importance of Multiple Banks**
    - Reduces probability of **bank conflicts**
        - Bank conflicts occur when multiple accesses target same bank
        - More banks help spread accesses across multiple banks
    - Supports required memory size
        - Cell array size limited by latency and manufacturability
        - Multiple banks needed to achieve total memory capacity
- **Thread Execution and DRAM Organization**
    - Sufficient threads needed for specified memory access bandwidth
    - **Benefits of Maximizing Occupancy**:
        - **Hides core pipeline latency**
        - Utilizes instruction throughput efficiently
        - Ensures enough memory requests to hide DRAM latency
        - Helps utilize memory bandwidth efficiently
    - **Requirements for Best Bandwidth**:
        - Even distribution across channels and banks
        - Coalesced bank access
- **Array Element Distribution Example**
    - Assumes 8-byte burst size (2 elements)



- **Distribution Pattern**:
  - M[0-1]: Channel 0, Bank 0
  - M[2-3]: Channel 1, Bank 0
  - M[4-5]: Channel 2, Bank 0
  - M[6-7]: Channel 3, Bank 0
  - M[8-9]: Channel 0, Bank 1
  - M[10-11]: Channel 1, Bank 1
  - M[12-13]: Channel 2, Bank 1
  - M[14-15]: Channel 3, Bank 1
- Pattern wraps back to Channel 0, Bank 0 for subsequent elements

- **Data Distribution Across Memory**
  - Elements are spread across banks and channels in the system
  - Even small arrays are distributed effectively
  - Elements fill DRAM burst of bank 0 channel 0 before moving to bank 0 channel 1
  - With 16+ elements, distribution uses all channels and banks
- **Parallel Thread Execution and Memory Organization**
  - Example uses **2 x 2 thread blocks** and **2 x 2 tiles**

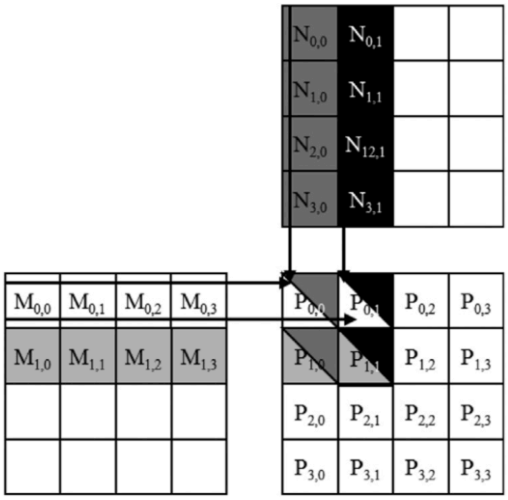


FIGURE 6.10

A small example of matrix multiplication (replicated from Fig. 5.5).

Tiles loaded by	Block 0,0	Block 0,1	Block 1,0	Block 1,1
Phase 0 (2D index)	M[0][0],M[0][1], M[1][0], M[1][1]	M[0][0], M[0][1], M[1][0], M[1][1]	M[2][0], M[2][1], M[3][0], M[3][1]	M[2][0], M[2][1], M[3][0], M[3][1]
Phase 0 (linearized index)	M[0], M[1], M[4], M[5]	M[0], M[1], M[4], M[5]	M[8], M[19], M[12], M[13]	M[8], M[9], M[12], M[13]
Phase 1 (2D index)	M[0][2],M[0][3], M[1][2], M[1][3]	M[0][2],M[0][3], M[1][2], M[1][3]	M[2][2], M[2][3], M[3][2], M[3][3]	M[2][2], M[2][3], M[3][2], M[3][3]
Phase 1 (linearized index)	M[2], M[3], M[6], M[7]	M[2], M[3], M[6], M[7]	M[10], M[11], M[14], M[15]	M[10], M[11], M[14], M[15]

FIGURE 6.11

M elements loaded by thread blocks in each phase.

- ### Phase 0 Execution:
  - All four thread blocks load first tile
  - Makes two coalesced accesses per block
  - Accesses parallel across two banks in channel 0 and channel 2
  - Block0,0 and Block0,1 load same M elements
  - Modern GPU caches combine these accesses when timing allows
- ### Cache Memory Role
  - Primarily designed to combine accesses
  - Reduces DRAM system access frequency
- ### Phase 1 Execution
  - Accesses occur in banks of channel 1 and channel 3
  - Maintains parallel access pattern

- ## Symbiotic Relationship: Threads and DRAM

- Good DRAM bandwidth utilization requires:
  - Many threads accessing data simultaneously
  - Effective use of parallel DRAM structure (banks/channels)
- Performance Impact:
  - Single channel access by all threads reduces memory throughput
  - Overall device execution speed depends on balanced channel usage

- **Scaling Considerations**
  - Larger matrices (8 x 8) with 2 x 2 thread blocks use all four channels
  - Increased DRAM burst size requires larger matrices for full bandwidth utilization

---

5.3 Thread coarsening

- **Thread Granularity in Kernels**
  - Previously seen kernels used **finest granularity** parallelization
    - Vector addition: one thread per output element
    - RGB-to-grayscale: one thread per output pixel
    - Image blur: one thread per output pixel
    - Matrix multiplication: one thread per output matrix element
- **Advantages of Finest Granularity**
  - Enables **transparent scalability**
  - When hardware has sufficient resources:
    - Application exposes enough parallelism for full hardware utilization
  - When hardware lacks resources:
    - Hardware can serialize work by executing thread blocks sequentially
- **Disadvantages of Finest Granularity**
  - Introduces **parallelization costs**:
    - Redundant data loading across thread blocks
    - Redundant work
    - Synchronization overhead
  - Costs are justified when threads run in parallel
  - Costs become unnecessary when work is serialized due to insufficient resources
- **Thread Coarsening Solution**
  - Assigns **multiple work units per thread**

Thread Coarsening in CUDA Programming

Thread coarsening is an optimization technique in CUDA programming where multiple logical threads are combined into a single physical thread. Each coarsened thread performs the work originally assigned to multiple threads. This approach can improve performance by reducing overhead associated with thread scheduling and memory accesses.

Why Use Thread Coarsening?

1. **Reduce Thread Management Overhead**: CUDA incurs some overhead in managing and scheduling threads. By reducing the total number of threads, these overheads are minimized.
2. **Improve Memory Access Patterns**: Coarsening can help improve memory coalescing or reduce contention by aggregating accesses within fewer threads.
3. **Better Utilization of Resources**: Sometimes, the hardware has more computational power than is utilized by the fine-grained thread distribution. Coarsening allows a single thread to better exploit the available computational resources.

How Thread Coarsening Works

In a typical CUDA kernel, each thread operates on one element of an array or performs a single computation. With thread coarsening:

- A single thread processes **multiple elements** or performs **multiple computations**.
- The index calculations are adjusted to accommodate this.

Example Without Thread Coarsening

cuda 复制代码

```cuda
__global__ void vectorAdd(float* A, float* B, float* C, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}
```

Here, each thread handles one element of the vectors `A`, `B`, and `C`.

Example With Thread Coarsening (Factor of 2)

cuda 复制代码

```cuda
__global__ void vectorAddCoarsened(float* A, float* B, float* C, int N) {
    int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 2; // Coarsening factor
    if (idx < N) {
```

```
        C[idx] = A[idx] + B[idx];
        if (idx + 1 < N) { // Ensure bounds are not exceeded
            C[idx + 1] = A[idx + 1] + B[idx + 1];
        }
    }
}
```

Key Adjustments:

1. **Indexing**: Adjust thread indices to cover multiple elements (`* 2` in this case).
2. **Bounds Checking**: Ensure threads don't exceed the array size.
3. **Workload Distribution**: Each thread does the work for two elements.

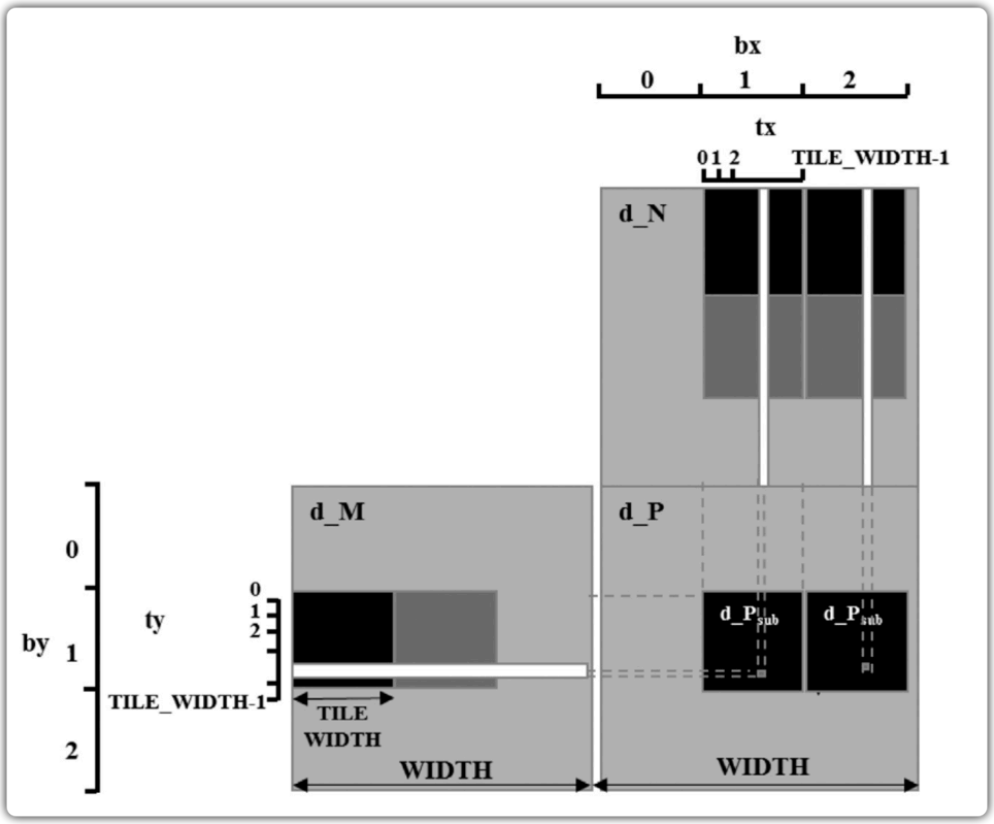
Advantages of Thread Coarsening

1. **Reduced Launch Overhead**: Fewer threads need to be launched, reducing kernel launch costs.
2. **Better Warp Utilization**: Threads within a warp can process more data, reducing idle cores.
3. **Improved Memory Access**: Helps align memory accesses or reduce shared memory contention.

Potential Drawbacks

1. **Reduced Parallelism**: Coarsening reduces the total number of threads, which may hurt performance if there are more computational units than threads.
2. **Increased Register Pressure**: Each thread handles more work, potentially increasing register usage and spilling.
3. **Complexity**: The kernel logic becomes more complicated, with additional indexing and bounds checks.

- Partially serializes work to reduce parallelization costs
- Example: Tiled Matrix Multiplication



- Adjacent output tiles share input tiles from matrix M
- Traditional approach: Different thread blocks load same M tiles redundantly
- Coarsened approach: Single thread block processes multiple output tiles
  - Loads M tiles once
  - Reuses data across multiple computations
  - Each thread handles multiple output elements

- **Thread Coarsening Implementation in Tiled Matrix Multiplication**
  - Introduces **COARSE_FACTOR** constant representing number of work units per thread

```
#define TILE_WIDTH 32
#define COARSE_FACTOR 4

__global__ void matrixMulKernel(float* M, float* N, float* P, int width) {
   __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
   __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];

   int bx = blockIdx.x;
   int by = blockIdx.y;
   int tx = threadIdx.x;
   int ty = threadIdx.y;

   // Identify the row and column of the P element to work on
   int row = by * TILE_WIDTH + ty;
   int colStart = bx * TILE_WIDTH * COARSE_FACTOR + tx;

   // Initialize Pvalue for all output elements
   float Pvalue[COARSE_FACTOR];
   for (int c = 0; c < COARSE_FACTOR; ++c) {
      Pvalue[c] = 0.0f;
   }

   // Loop over the M and N tiles required to compute P element
   for (int ph = 0; ph < width/TILE_WIDTH; ++ph) {
      // Collaborative loading of M tile into shared memory
      Mds[ty][tx] = M[row * width + ph * TILE_WIDTH + tx];
```

```
        for (int c = 0; c < COARSE_FACTOR; ++c) {
            int col = colStart + c * TILE_WIDTH;
            // Collaborative loading of N tile into shared memory
            Nds[ty][tx] = N[(ph * TILE_WIDTH + ty) * width + col];
        }
        __syncthreads();

        for (int k = 0; k < TILE_WIDTH; ++k) {
            Pvalue[c] += Mds[ty][k] * Nds[k][tx];
        }
        __syncthreads();
    }

    for (int c = 0; c < COARSE_FACTOR; ++c) {
        int col = colStart + c * TILE_WIDTH;
        P[row * width + col] = Pvalue[c];
    }
}
```

- Modifies **column index initialization**:
  - Uses **colStart** instead of simple column index
  - Calculation: **block index × TILE_WIDTH × COARSE_FACTOR**
- Multiple **Pvalue instances** declared for each element
- Uses **coarsening loops** to:
  - Iterate over work units
  - Load multiple N tiles per M tile
  - Update multiple output elements

- **Performance Benefits and Considerations**
  - Can provide **substantial performance improvements**
  - **Commonly applied optimization** in many applications
- **Key Pitfalls to Avoid**
  - **Unnecessary Application**
    - Only useful when there's a **price for parallelization**
    - Examples of valid costs:
      - Redundant data loading
      - Redundant work
      - Synchronization overhead
    - Not beneficial for:
      - Vector addition
      - RGB-to-grayscale conversion
  - **Over-coarsening**
    - Can lead to **underutilized hardware resources**
    - Reduces **exposed parallelism**
    - Best coarsening factor is:
      - **Device-specific**
      - **Dataset-specific**
      - Requires **retuning** for different scenarios
  - **Resource Consumption**

- May increase:
  - **Registers per thread**
  - **Shared memory per thread block**
- Must avoid **reducing occupancy**
- Performance loss from reduced occupancy may **outweigh coarsening benefits**

5.4 A checklist of optimizations

- **A checklist of optimizations**

Optimization	Benefit to compute cores	Benefit to memory	Strategies
Maximizing occupancy	More work to hide pipeline latency	More parallel memory accesses to hide DRAM latency	Tuning usage of SM resources such as threads per block, shared memory per block, and registers per thread
Enabling coalesced global memory accesses	Fewer pipeline stalls waiting for global memory accesses	Less global memory traffic and better utilization of bursts/cache lines	Transfer between global memory and shared memory in a coalesced manner and performing uncoalesced accesses in shared memory (e.g., corner turning); Rearranging the mapping of threads to data; Rearranging the layout of the data
Minimizing control divergence	High SIMD efficiency (fewer SIMD execution idle cores during)	-	Rearranging the mapping of threads to work and/or data; Rearranging the layout of the data
Tiling of reused data	Fewer pipeline stalls waiting for global memory accesses	Less global memory traffic	Placing data that is reused within a block in shared memory or registers so that it is transferred between global memory and the SM only once
Privatization (covered later)	Fewer pipeline stalls waiting for atomic updates	Less contention and serialization of atomic updates	Applying partial updates to a private copy of the data and then updating the universal copy when done
Thread coarsening	Less redundant work, divergence, or synchronization	Less redundant global memory traffic	Assigning multiple units of parallelism to each thread to reduce the price of parallelism when it is incurred unnecessarily

- **Key Optimizations in CUDA Programming**
  - Consolidation of optimizations into a checklist
  - Focus on universal optimizations across different applications

- Will be applied to parallel patterns and applications in later parts

- **Optimization 1: Maximizing Thread Occupancy on SMs**
  - Introduced in **Chapter 4: Compute Architecture and Scheduling**
  - **Key Points**:
    - Need more threads than cores
    - Helps hide long-latency operations
    - Requires careful tuning of resource usage
    - Must balance block and register usage per SM
  - **Benefits**:
    - Better thread assignment to SMs
    - Improved memory latency hiding
    - Fuller utilization of memory bandwidth
- **Optimization 2: Coalesced Global Memory Accesses**
  - **Core Concept**: Threads in same warp should access adjacent memory locations
  - **Benefits**:
    - Reduces global memory traffic
    - Improves DRAM burst utilization
  - **Implementation**:
    - Natural in basic kernels
    - More challenging with irregular memory patterns
    - Requires additional effort in complex applications
  - **Strategies for Coalescing in Applications with Irregular Access Patterns**
    - **Strategy 1: Load data from global to shared memory**
      - Example: Corner turning
      - Example: Merge pattern (Chapter 12)
        - Threads perform binary search in shared memory
      - Example: Sorting pattern (Chapter 13)
        - Scattered writes done in shared memory first
    - **Strategy 2: Rearrange thread mapping to data elements**
      - Example: Reduction pattern (Chapter 10)
    - **Strategy 3: Rearrange data layout**
      - Example: Sparse Matrix Computation (Chapter 14)
        - ELL and JDS formats
- **Minimizing Control Divergence Strategies**
  - **Strategy 1: Rearrange work/data distribution**
    - Example: Reduction pattern (Chapter 10)

        - Example: Scan pattern (Chapter 11)
        - Example: Graph Traversal (Chapter 15)
            - Tradeoffs between vertex-centric and edge-centric parallelization
    - **Strategy 2: Rearrange data layout**
        - Example: Sparse Matrix Computation (Chapter 14)
            - JDS format specifically
- **Tiling Data Optimization**
    - **Key Concepts**
        - Store reused data in shared memory or registers
        - Transfer data between global memory and SM only once
        - Applicable to both input and output data
    - **Applications and Challenges**
        - Matrix multiplication (Chapter 5)
        - Convolution pattern (Chapter 7)
            - Challenge: Different input/output tile dimensions
        - Stencil pattern (Chapter 8)
            - Notable use of register storage
- **Privatization**
    - Used when multiple threads/blocks need to update universal output
    - Creates **private copy** of data that can be partially updated
    - Final update made to universal copy when done
    - Examples:
        - Chapter 9 (Parallel Histogram) - updating histogram counters
        - Chapter 15 (Graph Traversal) - adding entries to same queue
- **Thread Coarsening**
    - Assigns multiple units of parallelism to a single thread
    - Reduces parallelism cost when hardware would serialize threads anyway
    - Applications across different patterns:
        - **Tiled Matrix Multiplication**
            - Reduces redundant input tile loading
            - One thread block processes multiple adjacent output tiles
        - **Stencil Pattern (Chapter 8)**
            - Reduces redundant input data loading
        - **Histogram Pattern (Chapter 9)**
            - Reduces number of private copies needing commitment
        - **Reduction Pattern (Chapter 10)**
            - Reduces synchronization and control divergence overhead
        - **Scan Pattern (Chapter 11)**
            - Reduces synchronization and control divergence overhead
            - Reduces redundant work compared to sequential algorithm
        - **Merge Pattern (Chapter 12)**
            - Reduces number of binary search operations
        - **Sort Pattern (Chapter 13)**
            - Improves memory coalescing

5.5 Knowing your computation’s bottleneck

- Understanding **Performance Optimization in GPU Computing**
  - **Key Concept**: Identifying the **performance bottleneck** - the resource limiting computation performance
  - **Optimization Strategy**:
    - Uses more of one resource to reduce burden on another
    - Must target the actual bottleneck resource
    - Incorrect optimization may result in **no benefit or worse performance**
- **Example Case: Shared Memory Tiling**
  - **Purpose**: Increases shared memory use to reduce global memory bandwidth pressure
  - **When Effective**:
    - Bottleneck is global memory bandwidth
    - Data being loaded is reused
  - **When Problematic**:
    - If performance is limited by occupancy
    - If shared memory usage is already constraining occupancy
- **Performance Analysis**
  - **Tools Available**:
    - GPU platforms provide various profiling tools
    - Refer to CUDA documentation for profiling guidance
  - **Important Considerations**:
    - Bottlenecks may be **hardware-specific**
    - Same computation may face different bottlenecks on different devices
    - Requires thorough understanding of:
      - GPU architecture
      - Architectural differences between GPU devices